DOCUMENTAÇÃO TÉCNICA OFICIAL

# Clyvo Vet — Arquitetura de Software

Mapeamento Exaustivo de Pacotes, Classes, Responsabilidades e Fluxos do Sistema


**Integrantes do Projeto (FIAP Sprint)**

**Maicon Douglas** — RM561279

**Gabriel (Gabs)** — RM561551

**Charlles** — RM566482

**Iago D.** — RM565708


**Repositório da Aplicação Java:** https://github.com/MCGI-PW/API-Clyvo-M_Vet

**Repositório de DevOps & Cloud:** https://github.com/MCGI-PW/DevOps-Clyvo-M_Vet

**Vídeo Demonstrativo no YouTube:** https://youtu.be/548lg3zc7Mw

**Stack Tecnológica:** Java 21 LTS | Spring Boot 3.5.4 | Spring Security 6 | Flyway | Oracle ANSI SQL / H2 | Thymeleaf

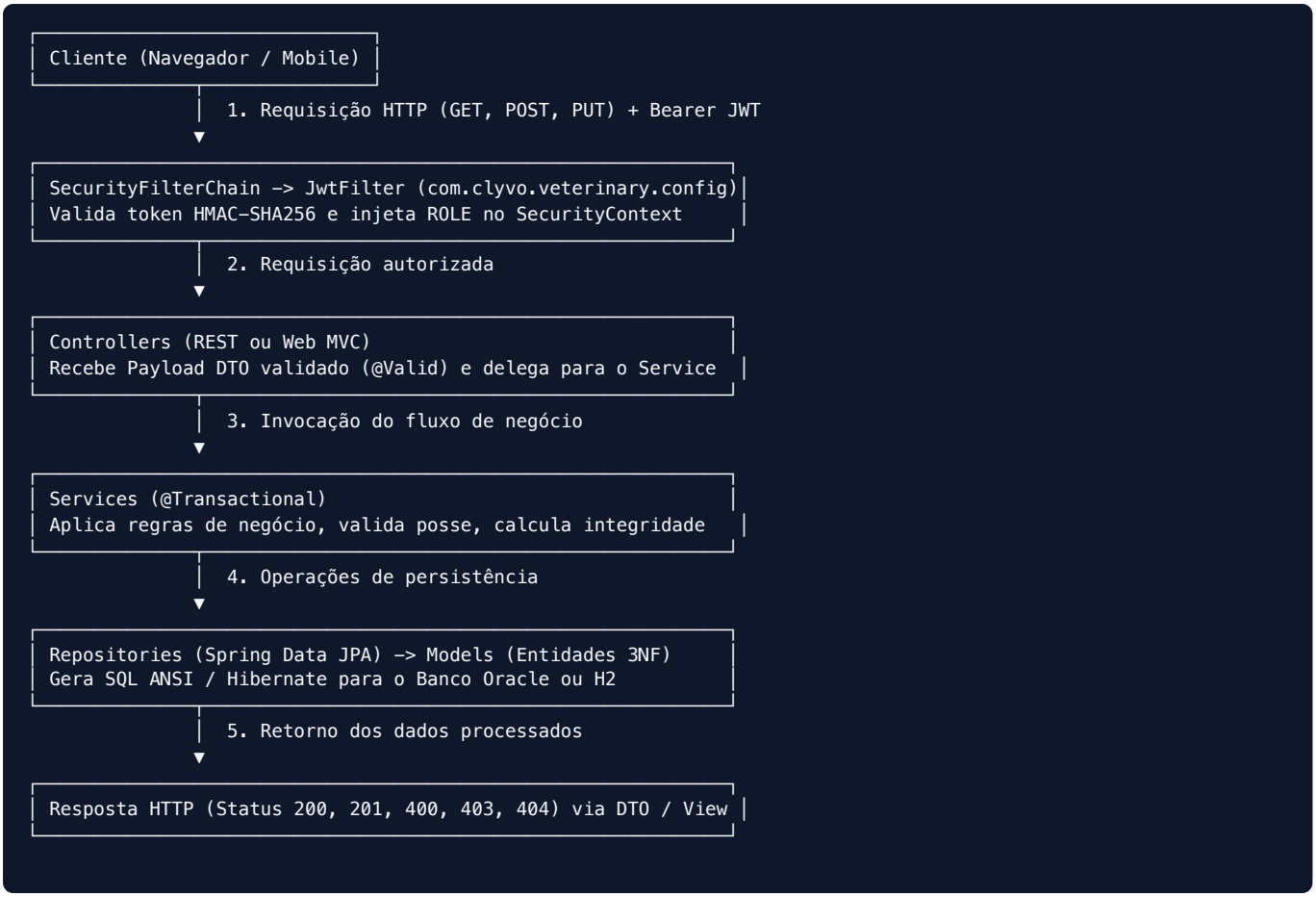
# 1. Fundamentos da Arquitetura em Camadas (Layered Architecture)

O sistema **Clyvo Vet** foi construído segundo o padrão clássico de **Arquitetura em Camadas (N-Tier Architecture)**, amplamente adotado em sistemas empresariais com Spring Boot. O objetivo primordial dessa abordagem é aplicar o princípio da **Separação de Preocupações (Separation of Concerns - SoC)** e a **Responsabilidade Única (SRP)** do SOLID, garantindo que cada componente de software desempenhe estritamente o papel que lhe cabe.

## O que cada camada faz e por que elas existem separadas:

- **Camada de Entrada e Apresentação (`controllers/` e `controllers/web/`):** É a fronteira de comunicação com o mundo externo. Sua única função é receber requisições HTTP, capturar cabeçalhos, rotas e payloads, acionar a camada de negócio correspondente e transformar o resultado em uma resposta HTTP apropriada (JSON para APIs REST ou View renderizada para o navegador). Controladores *jamais* devem conter cálculos, regras de domínio ou comandos diretos de SQL.
- **Camada de Transferência de Dados (`dto/`):** Isola o modelo interno do banco de dados (Entidades) do mundo externo. Evita o vazamento de campos sensíveis (como hashes BCrypt e contadores de segurança), previne vulnerabilidades de *Over-Posting* (quando o usuário tenta alterar campos protegidos como permissões de administrador) e centraliza a validação de entrada via *Bean Validation* (`@NotBlank`, `@Size`, `@Pattern`).
- **Camada de Regras de Negócio (`services/`):** É o núcleo inteligente da aplicação. Aqui são orquestrados os fluxos de trabalho, cálculos, controle transacional (`@Transactional`), validações de consistência lógica (ex: impedir agendamentos retroativos ou conflitos de agenda médica) e disparo de eventos/notificações.
- **Camada de Acesso a Dados (`repositories/`):** Responsável pela persistência e recuperação de dados. Utiliza o *Spring Data JPA* para abstrair o dialeto SQL subjacente, oferecendo métodos prontos para operações CRUD e permitindo consultas customizadas via *Derived Queries* ou JPQL sem acoplamento manual a conexões JDBC.
- **Camada de Domínio Relacional (`models/` e `models/enums/`):** Mapeia o modelo de dados relacional em 3ª Forma Normal (3NF) para classes Java via JPA/Hibernate. Define as tabelas, tipos de dados, chaves primárias e relacionamentos (`@OneToMany`, `@ManyToOne`, `@JoinColumn`), além de enums que garantem integridade de tipos.
- **Camada de Tratamento de Falhas (`exceptions/`):** Intercepta exceções lançadas pelas camadas inferiores de forma centralizada e padronizada (`@RestControllerAdvice`), gerando respostas HTTP sem expor detalhes internos de implementação nem stack traces do servidor.
- **Camada de Infraestrutura e Configuração (`config/`):** Centraliza configurações transversais: cadeia de filtros de segurança (Spring Security), decodificação de tokens JWT, documentação Swagger/OpenAPI, internacionalização (i18n) e carga inicial de dados em ambientes controlados.

## Diagrama do Ciclo de Vida de uma Requisição


Cliente (Navegador / Mobile)
1. Requisição HTTP (GET, POST, PUT) + Bearer JWT
SecurityFilterChain -> JwtFilter (com.clyvo.veterinary.config)
Valida token HMAC-SHA256 e injeta ROLE no SecurityContext
2. Requisição autorizada
Controllers (REST ou Web MVC)
Recebe Payload DTO validado (@Valid) e delega para o Service
3. Invocação do fluxo de negócio
Services (@Transactional)
Aplica regras de negócio, valida posse, calcula integridade
4. Operações de persistência
Repositories (Spring Data JPA) -> Models (Entidades 3NF)
Gera SQL ANSI / Hibernate para o Banco Oracle ou H2
5. Retorno dos dados processados
Resposta HTTP (Status 200, 201, 400, 403, 404) via DTO / View

## 2. Arquivos da Raiz do Projeto e Infraestrutura de Build

### `pom.xml`

O arquivo **Project Object Model (POM)** gerencia todas as dependências, plugins de compilação e propriedades do Maven. Principais definições:

- **Java 21 LTS:** Configurado com `<java.version>21</java.version>` e a tag `<release>21</release>` no `maven-compiler-plugin` para garantir estrita conformidade com o bytecode do Java 21.
- **Spring Boot Starter Web & Validation:** Fornece o servidor Tomcat embarcado, suporte a REST/MVC e Bean Validation (Hibernate Validator).
- **Spring Boot Starter Security:** Habilita autenticação, autorização baseada em papéis (RBAC) e proteção de endpoints.
- **Spring Boot Starter Data JPA & Flyway:** Camada ORM com Hibernate e controle de versões de banco de dados via migrations versionadas.
- **Drivers de Banco:** `ojdbc11` (Oracle Database da FIAP) e `h2` (banco em memória para perfil local e testes).
- **JJWT (io.jsonwebtoken):** Biblioteca para criação, decodificação e validação criptográfica de tokens JWT (HS256).
- **SpringDoc OpenAPI (springdoc-openapi-starter-webmvc-ui):** Geração automática da documentação Swagger UI interativa em `/swagger-ui.html`.
- **Spring Boot Starter Thymeleaf:** Motor de renderização de templates HTML server-side com suporte a i18n e layout responsivo.

### `mvnw e mvnw.cmd`

Scripts do **Maven Wrapper** para Unix/macOS e Windows. Baixam e executam a versão homologada do Maven (3.9.x) de forma hermética, permitindo que qualquer desenvolvedor execute comandos de build (`./mvnw compile`, `./mvnw test`, `./mvnw spring-boot:run`) sem necessidade de instalar o Maven no sistema operacional.

### `README.md`

Documentação primária do repositório contendo visão geral da solução, checklist de atendimento aos requisitos, tabela de migrations Flyway, matriz de rotas, instruções de execução, integrantes do projeto e link para o vídeo oficial no YouTube.

### `ClyvoVeterinaryApplication.java (com.clyvo.veterinary)`

Ponto de entrada (*entrypoint*) da aplicação. Contém o método `public static void main(String[] args)` que invoca `SpringApplication.run(ClyvoVeterinaryApplication.class, args)`. A anotação `@SpringBootApplication` combina `@Configuration`, `@EnableAutoConfiguration` e `@ComponentScan`, escaneando e registrando automaticamente todos os componentes dentro do pacote base.

## 3. Pacote com.clyvo.veterinary.config (Segurança e Infraestrutura)

### `SecurityConfig.java`

Classe mestre de segurança da aplicação, anotada com `@Configuration` e `@EnableWebSecurity`. Suas responsabilidades incluem:

- **Controle de Rotas por Perfil (RBAC):** Define quais perfis têm acesso a quais rotas:
  - Rotas de Tutores (`/api/pets/**`, `POST /api/consultas`): exigem autoridade `ROLE_TUTOR`.
  - Rotas de Médicos (`/api/appointments/**`, conclusão médica): exigem autoridade `ROLE_VETERINARIO`.
  - Rotas Administrativas (`/api/clinicas/**`): exigem autoridade `ROLE_CLINICA`.
  - Rotas Públicas: `/api/auth/**`, `/web/login`, `/swagger-ui/**`, `/v3/api-docs/**`, `/h2-console/**` e arquivos estáticos.
- **Injeção do Filtro JWT:** Posiciona o `JwtFilter` antes do filtro de autenticação padrão do Spring (`UsernamePasswordAuthenticationFilter`).
- **Criptografia Forte:** Registra o bean `PasswordEncoder` implementado por `BCryptPasswordEncoder(12)`.
- **Sessão Flexível:** Configura a política de criação de sessão como `IF_REQUIRED`, permitindo tanto chamadas stateless via JWT quanto sessões web para o Thymeleaf.

### `JwtFilter.java`

Componente anotado com `@Component` que herda de `OncePerRequestFilter`. Intercepta cada requisição HTTP recebida pela aplicação, lê o cabeçalho `Authorization` procurando pelo prefixo `Bearer `, valida a assinatura digital e o prazo de validade via `JwtUtil`, extrai a role do usuário e injeta um `UsernamePasswordAuthenticationToken` autenticado no `SecurityContextHolder` do Spring Security.

### `JwtUtil.java`

Classe utilitária para gestão de JSON Web Tokens utilizando a biblioteca JJWT. Responsável por:

- Assinar os tokens digitalmente usando o algoritmo criptográfico HMAC-SHA256 a partir da chave configurada (`jwt.secret`).
- Inserir *claims* customizadas no payload: `userId`, `email`, `role` e `idEntidade` (ID específico da tabela Tutor, Veterinário ou Clínica).
- Validar integridade e tempo de expiração do token (definido em 24 horas por padrão).

### `InternacionalizacaoConfig.java`

Configura o suporte multilíngue da aplicação implementando a interface `WebMvcConfigurer`. Expõe dois beans fundamentais:

- `SessionLocaleResolver`: Define o idioma padrão da aplicação como Português do Brasil (`pt_BR`).
- `LocaleChangeInterceptor`: Intercepta o parâmetro `lang` na URL (exemplo: `?lang=en`) e atualiza automaticamente a localidade ativa na sessão HTTP.

### `SwaggerConfig.java`

Configura a especificação OpenAPI 3.0 para documentação interativa da API. Define título, descrição, versão e registra o esquema global de segurança **bearerAuth** (JWT Bearer Token), permitindo que desenvolvedores autentiquem e testem requisições protegidas diretamente pela interface gráfica do Swagger.

### `DataInitializer.java`

Componente anotado com `@Component` e `@Profile({"local", "dev"})` que implementa `CommandLineRunner`. Ao iniciar a aplicação em ambiente de desenvolvimento/local, verifica a existência de dados e popula contas de demonstração completas com senhas hashificadas por BCrypt:

- **Tutor:** `tutor@teste.com` / 123456 (com cão Rex e gato Mingau).
- **Veterinário:** `vet@teste.com` / 123456 (CRMV-SP 12345).
- **Clínica:** `clinica@teste.com` / 123456 (CNPJ e equipe vinculada).

> **Proteção de Produção**
> Por utilizar a anotação `@Profile`, o `DataInitializer` nunca é executado quando a aplicação sobe apontando para o banco de dados oficial da faculdade, garantindo que senhas de teste não sejam criadas em ambientes reais.

## 4. Camada de Controladores (REST e Web MVC)

O projeto divide seus controladores em duas categorias bem delineadas:

- **Controladores REST (`controllers/`):** Anotados com `@RestController`. Destinados ao consumo por aplicações cliente (como o App Mobile Flutter ou integrações externas), serializam e deserializam dados diretamente em formato JSON.
- **Controladores Web (`controllers/web/`):** Anotados com `@Controller`. Destinados à interface do usuário no navegador via Thymeleaf, utilizam o padrão **Post-Redirect-Get (PRG)** com `RedirectAttributes` para prevenir reenvios acidentais de formulário ao pressionar F5.

### Controladores REST (API JSON)

#### `AuthController.java`

Centraliza o fluxo de autenticação e registro público da API REST:

- `POST /api/auth/login`: Recebe `AuthRequest` (email/senha), valida através do `AuthService` e retorna `AuthResponse` contendo o token JWT.
- `POST /api/auth/register`: Recebe `RegisterRequest`, valida restrições de e-mail duplicado e cria a conta no perfil desejado.

#### `PetController.java`

Gerencia as operações dos animais vinculados ao tutor autenticado:

- `GET /api/pets`: Retorna lista de pets pertencentes ao tutor logado. Protegido com `@PreAuthorize("hasRole('TUTOR')")`.
- `POST /api/pets`: Cadastra um novo animal associado ao tutor logado.

#### `ConsultaController.java`

Controla os agendamentos e cancelamentos de consultas veterinárias:

- `POST /api/consultas`: Agenda uma nova consulta com validações de horário e conflito médico.
- `GET /api/consultas`: Lista consultas ativas ou históricas do usuário solicitante.
- `PUT /api/consultas/{id}/cancelar`: Cancela um agendamento futuro e notifica os participantes.

#### `AppointmentController.java`

Controlador especializado no atendimento clínico médico. Disponibiliza a rota médica:
`POST /api/appointments/{id}/complete`: Conclui formalmente a consulta e persiste o **Prontuário Médico** correspondente. Restrito a `ROLE_VETERINARIO`.

#### `ClinicaController.java`

Operações de gestão clínica e multi-tenancy:

- `GET /api/clinicas`: Catálogo público de clínicas credenciadas.
- `GET /api/clinicas/{id}/veterinarios`: Lista o corpo médico atuante na clínica especificada.
- `POST /api/clinicas/veterinarios/vincular`: Vincula um veterinário à equipe médica da clínica.
- `PUT /api/clinicas/autorizacoes/{id}/transferir`: Transfere consentimento e consultas de um médico para outro.

**`AutorizacaoController.java`**

Governança de privacidade e consentimento de dados médicos (LGPD). Fornece `GET /api/autorizacoes` para consulta de autorizações concedidas e `PUT /api/autorizacoes/{id}/revogar` para cancelamento imediato de acesso pelo tutor.

**`VeterinarioController.java, TutorController.java, RacaController.java e NotificacaoController.java`**

Fornecem consultas de perfil (`/api/veterinarios`, `/api/tutores/me`), catálogo hierárquico de espécies e raças (`/api/especies`, `/api/racas`) e central de avisos (`/api/notificacoes`).

## Controladores Web MVC (Interface Thymeleaf)

**`WebAuthController.java`**

Controla a renderização da tela de login (`/web/login` e rota raiz `/`), processamento do formulário de autenticação via sessão HTTP com redirecionamento dinâmico para o dashboard do perfil correspondente e encerramento de sessão em `/web/logout`.

**`WebTutorController.java`**

Atende a rota `/web/tutor/dashboard`. Carrega a lista de animais do tutor, médicos disponíveis e histórico de consultas. Processa requisições POST para adição de pets e agendamento de consultas via PRG com mensagens flash de sucesso ou erro.

**`WebVetController.java`**

Atende a rota `/web/vet/dashboard`. Exibe a fila médica de atendimentos do dia. Processa o formulário de finalização da consulta médica via `POST /web/vet/consultas/{id}/concluir`, persistindo diagnóstico, medicamentos prescritos e gerando o prontuário.

**`WebClinicaController.java`**

Atende a rota `/web/clinica/dashboard`. Fornece interface administrativa para acompanhamento do corpo clínico, visualização das consultas realizadas na unidade e suporte à transferência de atendimentos entre profissionais.

## 5. Camada de Objetos de Transferência de Dados (DTOs)

Os DTOs garantem que o contrato externo da API seja independente da estrutura do banco de dados relacional. Cada classe DTO define anotações do *Jakarta Bean Validation* para garantir a sanitização dos dados antes de atingirem a lógica de negócio:

| DTO | Campos e Tipos | Anotações de Validação | Propósito Arquitetural |
|---|---|---|---|
| **AuthRequest** | `identificador` (String)<br>`senha` (String) | `@NotBlank` em ambos os campos | Recebe credenciais de login (e-mail e senha) de forma padronizada. |
| **AuthResponse** | `token` (String)<br>`tipo` (String)<br>`tipoConta` (TipoConta)<br>`nome` (String)<br>`idEntidade` (String) | N/A (DTO de saída) | Devolve o token JWT gerado e dados contextuais para a interface ou app mobile. |
| **RegisterRequest** | `email`, `senha`, `nome`, `tipoConta`, `crmv`, `telefone`, `cnpj`, `razaoSocial` | `@NotBlank`, `@Email`, `@Size(min=6)`, `@NotNull` | Captura dados cadastrais polimórficos, aplicando validações específicas para cada tipo de usuário. |
| **ScheduleAppointmentRequest** | `petId` (String)<br>`veterinarioId` (String)<br>`clinicaId` (String)<br>`dataHora` (LocalDateTime)<br>`motivo` (String) | `@NotBlank`, `@NotNull`, `@Future`, `@Size(max=500)` | Recebe dados para agendamento, impedindo datas passadas e campos vazios. |
| **CompleteAppointmentRequest** | `diagnostico` (String)<br>`prescricao` (String)<br>`observacoesClinicas` (String) | `@NotBlank(diagnostico)`, `@Size(max=4000)` | Captura o fechamento clínico preenchido pelo médico veterinário para gerar o prontuário. |

## 6. Camada de Domínio e Modelo Relacional 3NF (models e enums)

O modelo de dados foi projetado em **Terceira Forma Normal (3NF)**, eliminando redundâncias e dependências transitivas. A estrutura utiliza chaves primárias binárias de 16 bytes (`RAW(16)` em Oracle, mapeadas como `String` com formatação hexadecimal em Java).

### Subpacote models/enums/ (Tipos Fortemente Tipados)

- `TipoConta`: Define os papéis de acesso do sistema: `TUTOR`, `VETERINARIO` e `CLINICA`.
- `StatusConta`: Estado de ativação: `ATIVO`, `BLOQUEADO`, `PENDENTE`.
- `StatusConsulta`: Máquina de estados da consulta: `AGENDADA`, `EM_ANDAMENTO`, `CONCLUIDA`, `CANCELADA`.
- `ModalidadeConsulta`: Tipo de atendimento: `PRESENCIAL` ou `TELEMEDICINA`.
- `StatusAutorizacao`: Ciclo de vida do consentimento LGPD: `ATIVA`, `REVOGADA`, `EXPIRADA`.
- `StatusVinculo`: Estado do médico junto a uma clínica: `ATIVO`, `INATIVO`.

### Entidades JPA (models/)

- **ContaAcesso:** Entidade base de autenticação, contendo status da conta e data de registro.
- **Credencial:** Armazena o hash criptográfico BCrypt da senha, o contador de falhas consecutivas (`tentativasFalhas`) e o timestamp de suspensão temporária (`bloqueadoAte`).
- **IdentificadorAcesso:** Armazena a chave de busca para login (e-mail principal verificado).
- **Perfil e ContaPerfil:** Mapeiam os papéis RBAC atribuídos a cada conta de acesso.
- **Tutor:** Dados cadastrais do tutor de animais (nome, CPF, telefone, endereço).
- **Veterinario:** Cadastro do médico veterinário credenciado com CRMV e especialidade.
- **Clinica:** Registro da clínica ou hospital veterinário com CNPJ e razão social.
- **VeterinarioClinica:** Tabela associativa 3NF que mapeia os médicos vinculados a cada estabelecimento.
- **Especie e Raca:** Catálogo normalizado de taxonomia animal, garantindo integridade referencial.
- **Pet:** Registro do animal contendo nome, data de nascimento, peso, espécie e raça, vinculado ao tutor proprietário.
- **Consulta:** Entidade que orquestra o agendamento médico, conectando Pet, Tutor, Veterinário e Clínica, com controle rigoroso de data/hora e status.
- **Prontuario:** Registro médico oficial e imutável. Criado na conclusão da consulta, armazena diagnóstico clínico, prescrição de medicamentos, observações e data de emissão.
- **AutorizacaoAcessoPet:** Registro de consentimento explícito do tutor para visualização de prontuários por médicos/clínicas (conformidade LGPD).
- **Notificacao:** Registro histórico de avisos e alertas do sistema direcionados aos usuários.
- **Sessao:** Controle de sessões de acesso ativas por dispositivo e IP.

## 7. Camada de Acesso a Dados (repositories)

Todas as interfaces deste pacote estendem `JpaRepository<Entidade, TipoID>` do Spring Data JPA. Isso disponibiliza nativamente operações como `save()`, `findById()`, `findAll()`, `delete()` e suporte a paginação sem que seja necessário escrever uma única linha de SQL manual para operações triviais.

### Mapeamento dos 16 Repositórios do Sistema:

| Interface de Repositório | Entidade Gerenciada | Consultas Notáveis / Derived Queries |
|---|---|---|
| **ContaAcessoRepository** | ContaAcesso | `findByStatusConta(StatusConta status)` |
| **CredencialRepository** | Credencial | `findByContaAcessoId(String idConta)` |
| **IdentificadorAcessoRepository** | IdentificadorAcesso | `findByValor(String email)` — Chave do fluxo de autenticação. |
| **PerfilRepository** | Perfil | `findByNome(String nome)` |
| **TutorRepository** | Tutor | `findByContaAcessoId(String idConta)` |
| **VeterinarioRepository** | Veterinario | `findByCrmv(String crmv)`, `findByAtivoTrue()` |
| **ClinicaRepository** | Clinica | `findByCnpj(String cnpj)`, `findByAtivoTrue()` |
| **VeterinarioClinicaRepository** | VeterinarioClinica | `findByClinicaIdClinicaAndStatus(String id, StatusVinculo st)` |
| **PetRepository** | Pet | `findByTutorIdTutor(String idTutor)` — Garante isolamento entre tutores. |
| **EspecieRepository** | Especie | `findAllByOrderByNomeAsc()` |
| **RacaRepository** | Raca | `findByEspecieIdEspecieOrderByNomeAsc(String idEspecie)` |
| **ConsultaRepository** | Consulta | `findByTutorIdTutorOrderByDataHoraDesc(...)` `findConflitoAgendamento(...)`: Query customizada que impede choque de horários médicos em intervalos menores que 30 minutos. |
| **ProntuarioRepository** | Prontuario | `findByConsultaIdConsulta(String idConsulta)` `findByConsultaPetIdPetOrderByDataRegistroDesc(...)`: Histórico médico completo do animal. |
| **AutorizacaoAcessoPetRepository** | AutorizacaoAcessoPet | `findByPetAndVeterinarioAndStatus(...)`: Verificação de acesso para LGPD. |
| **NotificacaoRepository** | Notificacao | `findByContaAcessoOrderByDataEnvioDesc(...)` |
| **SessaoRepository** | Sessao | `findByContaAcessoIdAndRevogadoFalse(...)` |

## 8. Camada de Regras de Negócio e Serviços (services)

A camada de serviços é onde o comportamento do sistema é orquestrado. Todos os métodos que alteram estado no banco são anotados com `@Transactional`, garantindo atomicidade (ACID): ou a transação é completada com sucesso, ou sofre rollback integral caso ocorra uma exceção de negócio.

### `AuthService.java`

Responsável pelo ciclo de vida de acesso e segurança de identidades:

- **Prevenção Anti-Força Bruta:** Em caso de senha incorreta, incrementa o contador de tentativas da credencial. Ao atingir 5 erros consecutivos, bloqueia a conta por 15 minutos (`AccountBlockedException`). Caso a autenticação seja bem-sucedida, o contador de falhas é zerado.
- **Proteção contra Enumeração de Usuários:** Tanto se o e-mail não existir quanto se a senha estiver errada, a mensagem retornada é padronizada (*"E-mail ou senha incorretos."*).
- **Cadastro Atômico:** Cria de forma transacional o registro central de identidade (`ContaAcesso`), o identificador de e-mail (`IdentificadorAcesso`), a credencial criptografada com BCrypt (`Credencial`) e a entidade específica do perfil (`Tutor`, `Veterinario` ou `Clinica`).

### `ConsultaService.java`

Gerencia os dois fluxos de negócio mais importantes da aplicação:

- **Fluxo 1 — Agendamento de Consulta:** Valida se o pet pertence de fato ao tutor solicitante; rejeita agendamentos com data retroativa; consulta o `ConsultaRepository` para checar se o veterinário já possui outra consulta com sobreposição de horário; salva o agendamento no estado `AGENDADA`; cria automaticamente o termo de consentimento LGPD (`AutorizacaoAcessoPet`); e dispara notificações para o tutor e o médico.
- **Fluxo 2 — Conclusão Médica e Emissão de Prontuário:** Valida se o usuário autenticado é o médico responsável; valida se a consulta encontra-se no estado `AGENDADA` ou `EM_ANDAMENTO`; atualiza o status da consulta para `CONCLUIDA`; cria e persiste o registro imutável em `Prontuario` contendo diagnóstico, prescrição e observações; e emite notificações de encerramento do atendimento ao tutor.
- **Cancelamento de Consulta:** Permite cancelar consultas agendadas, atualizando o status para `CANCELADA` e notificando as partes envolvidas.

### `AutorizacaoService.java`

Implementa a governança de acesso aos prontuários em conformidade com a LGPD. Permite ao tutor visualizar consentimentos ativos e revogá-los a qualquer momento, bloqueando o acesso médico retroativo.

### `ClinicaService.java`

Gerencia a alocação do corpo clínico. Permite que a gestão da clínica transfira prontuários e consultas de um médico para outro da mesma equipe médica sem perder o histórico do paciente.

### `PetService.java, VeterinarioService.java, TutorService.java, RacaService.java e NotificacaoService.java`

Serviços auxiliares que implementam validações de integridade cadastral de pets, busca por CRMV/especialidade, leitura/escrita de notificações e carregamento dinâmico de raças baseadas na espécie selecionada.

## 9. Camada de Tratamento Centralizado de Exceções (exceptions)

O tratamento de falhas segue as melhores práticas da RFC 7807 (*Problem Details for HTTP APIs*), garantindo que nenhuma exceção de runtime retorne erro 500 genérico ou detalhes da arquitetura interna.

### `GlobalExceptionHandler.java`

Classe anotada com `@RestControllerAdvice`. Intercepta todas as exceções lançadas nos controladores e gera uma resposta uniforme contendo:

```
{
  "timestamp": "2026-09-17T20:45:00",
  "status": 400,
  "error": "Bad Request",
  "message": "Horário indisponível para o médico veterinário selecionado."
}
```

| Classe da Exceção | Código HTTP Mapeado | Cenário de Disparo Típico |
|---|---|---|
| **ResourceNotFoundException** | `404 Not Found` | Busca por Pet, Consulta ou Usuário cujo ID não existe no banco de dados. |
| **BusinessException** | `400 Bad Request` | Violação de regras de negócio (conflito de agenda, agendamento no passado). |
| **AccessDeniedException** | `403 Forbidden` | Tutor tentando agendar consulta para pet que não é de sua propriedade. |
| **AccountBlockedException** | `403 Forbidden` | Tentativa de login em conta suspensa temporariamente por anti-força bruta. |
| **MethodArgumentNotValidException** | `400 Bad Request` | Falha na validação de campos anotados com `@Valid` (ex: e-mail inválido). |

## 10. Recursos da Aplicação (src/main/resources)

### Configurações de Ambiente

- **application.properties:** Perfil padrão de execução, preparado para o **Oracle Database da FIAP**. Utiliza variáveis de ambiente seguras (`${DB_URL}`, `${DB_USER}`, `${DB_PASS}`, `${JWT_SECRET}`), dialeto `org.hibernate.dialect.OracleDialect` e ativa o Flyway com `baseline-on-migrate=true`.
- **application-local.properties:** Perfil para desenvolvimento local e demonstração. Utiliza o banco de dados H2 em memória emulado para Oracle (`MODE=Oracle`), ativa o console web em `/h2-console` e dispensa configuração de banco externo.

### Controle de Versão do Banco de Dados (Flyway Migrations)

| Versão / Arquivo | Responsabilidade DDL |
|---|---|
| `V1__Create_3NF_Core.sql` | Criação do core relacional: contas de acesso, credenciais, perfis, vínculos e sessões. |
| `V2__Create_Racas.sql` | Criação das tabelas `especie`, `raca` e `pet`, com seed de mais de 30 raças. |
| `V3__Create_Consulta_And_Populate_Vets.sql` | Criação da tabela `consulta` e carga inicial de médicos veterinários credenciados. |
| `V4__Create_Notificacao.sql` | Criação da tabela `notificacao` para mensageria e alertas. |
| `V5__Add_Clinica_To_Consulta_And_Autorizacao.sql` | Adiciona `id_clinica` em consultas e suporte multi-tenancy para clínicas. |
| `V6__Create_Prontuario.sql` | Criação da tabela `prontuario` normalizada em 3NF vinculada à consulta e ao médico. |

### Internacionalização (i18n)

- **messages_pt_BR.properties:** Dicionário de mensagens em Português do Brasil de todos os botões, títulos de formulários, cabeçalhos de tabelas e alertas do sistema.
- **messages_en.properties:** Dicionário de mensagens traduzidas para o Inglês, alternáveis dinamicamente via parâmetro de URL (`?lang=en`).

### Camada Visual (Templates Thymeleaf e Static)

- `fragments/header.html`: Cabeçalho unificado com logotipo, seletor dinâmico de idiomas e identificação do usuário logado.
- `fragments/footer.html`: Rodapé padrão institucional.
- `web/login.html`: Tela inicial contendo formulário de login e alternância para cadastro.
- `web/dashboard-tutor.html`: Painel do tutor com gestão de pets e agendamento de consultas.
- `web/dashboard-vet.html`: Painel médico com fila de consultas e modal para fechamento de prontuário.
- `web/dashboard-clinica.html`: Painel da clínica com controle de equipe médica e transferências.
- `style.css` e `css/main.css`: Folhas de estilo modernas baseadas em **Aurora Glassmorphism** com alto contraste e design responsivo.
- `app.js`: Utilitários JavaScript para controle de modais e requisições assíncronas no frontend.

## 11. Suíte de Testes Automatizados (src/test)

A suíte de testes do projeto conta com **28 testes automatizados** divididos entre testes unitários (com Mockito) e testes de integração de controladores (com Spring MockMvc), cobrindo regras críticas de negócio e segurança.

| Classe de Teste | Tipo de Teste | Comportamentos Validados |
|---|---|---|
| **ClyvoVeterinaryApplicationTests** | Context Load | Garante que o Spring Boot inicializa todos os beans sem falhas de injeção de dependência. |
| **AuthServiceTest** | Unitário (Mockito) | Valida criptografia BCrypt, geração de tokens JWT e bloqueio de conta após 5 falhas no anti-brute force. |
| **ConsultaServiceTest** | Unitário (Mockito) | Testa agendamento válido, bloqueio de conflito de horários médicos e emissão correta do Prontuário Médico. |
| **PetServiceTest** | Unitário (Mockito) | Testa cadastro de pets, associação ao tutor autenticado e isolamento entre usuários. |
| **ConsultaControllerTest** | Integração (MockMvc) | Verifica rotas REST de consulta, códigos de status HTTP e regras de autorização do JWT. |
| **AutorizacaoControllerTest** | Integração (MockMvc) | Valida concessão e revogação de consentimento médico LGPD. |
| **ClinicaControllerTest** | Integração (MockMvc) | Testa endpoints administrativos de gestão de equipe médica e transferência de consultas. |

> **Comando para Execução dos Testes**
>
> Para rodar a suíte completa de testes no terminal do projeto, execute:
> `./mvnw test`